DAY 22

# Databricks Auto Loader

*Naye Files Ko Khud-Ba-Khud Pakadna*

**What you will learn**

How Auto Loader discovers files, tracks progress, infers and evolves schemas, handles unexpected data, uses checkpoints for exactly-once ingestion, and how to design a production-ready ingestion pattern.

**From Zero to Databricks MVP — Deep Dive Notes**

Verified against Databricks documentation current to September 2026

# 1. Why Auto Loader Exists

Imagine a cloud storage folder receiving thousands or millions of JSON, CSV, Parquet, XML, or Avro files over time. A normal batch job can read the folder, but every run still needs a reliable way to answer a deceptively hard question: which files are new, and which ones have already been processed?

Auto Loader solves this file-ingestion problem. It is a Databricks capability built on Spark Structured Streaming that incrementally discovers new files in cloud object storage and processes them through the cloudFiles source.

**First-principles idea**

Auto Loader is not a new storage system. Your files stay in cloud storage. Auto Loader is the ingestion mechanism that discovers new files, remembers what it has already seen, and feeds those files incrementally into a Spark streaming pipeline.

## Without Auto Loader

- Your code might repeatedly list an entire folder to find new files.
- You need your own state table or bookkeeping to remember processed files.
- Schema changes can break pipelines unless you handle them explicitly.
- Large folders can make repeated LIST operations slower and more expensive.

## With Auto Loader

- Databricks handles incremental file discovery.
- Checkpoint state tracks progress and discovered files.
- Schema inference and schema evolution can be managed automatically.
- File events can avoid repeatedly scanning very large source directories.

# 2. Auto Loader in One Picture

**Conceptual architecture**

```
Producer / Source System
        |
        v
Cloud Object Storage / External Volume
        |
        |  new files arrive
        v
+--------------------------+
|  Auto Loader (cloudFiles) |
|  - file discovery         |
|  - schema tracking        |
|  - file state tracking    |
+--------------------------+
        |
        v
Spark Structured Streaming
        |
        v
Bronze Delta Table
        |
        v
Silver / Gold transformations
```

The source path can be a cloud path or, in Unity Catalog-based designs, a volume path. Auto Loader produces a streaming DataFrame. The downstream sink is commonly a Delta table, but the central idea is incremental ingestion, not a specific medallion layer.

# 3. The cloudFiles Source

Auto Loader is invoked by using format("cloudFiles") on spark.readStream. You then tell Auto Loader the underlying file format with cloudFiles.format.

### Minimal PySpark pattern

```
source = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/Volumes/main/ops/meta/orders_schema")
    .load("/Volumes/main/ops/landing/orders"))

(source.writeStream
    .option("checkpointLocation", "/Volumes/main/ops/checkpoints/orders")
    .toTable("main.bronze.orders"))
```

| Piece | Purpose |
|---|---|
| format("cloudFiles") | Selects Auto Loader as the streaming source. |
| cloudFiles.format | Specifies the actual file format, such as json, csv, parquet, avro, or xml. |
| schemaLocation | Stores Auto Loader schema metadata over time. |
| checkpointLocation | Stores stream progress and file-tracking state. |
| load(path) | Source directory or volume containing incoming files. |
| toTable(...) | Writes the streaming result to a managed Delta table. |

# 4. How Auto Loader Finds New Files

Auto Loader supports multiple file-discovery approaches. The important production distinction in current Databricks guidance is directory listing versus file notification mode, especially managed file events.

| Mode | How it discovers files | Strength | Trade-off / use case |
|---|---|---|---|
| Directory listing | Lists the input directory to discover new files. | Simple; minimal extra cloud setup. | Repeated listing can become expensive and slower as directory size grows. |
| File notification — managed file events | Uses cloud-provider file event notifications and a Databricks-managed cache for file metadata. | More performant and scalable; recommended for most workloads. | Requires an external location and file-events prerequisites. |
| Classic file notifications | Uses cloud notification + queue resources associated with the stream. | Scalable and avoids full listing each trigger. | More cloud-specific resource management than managed file events. |

## Directory Listing: the Guard Analogy

Think of a security guard walking through the entire building again and again to see who is new. It is straightforward, but the work grows as the building gets bigger. Directory listing behaves similarly: it inspects the source directory to identify new files.

### Managed File Events: the Receptionist Analogy

Now imagine a receptionist who receives an event whenever a new visitor arrives. Auto Loader can use cloud file events so subsequent runs read file metadata from the file-events cache instead of repeatedly listing the whole directory.

**Current recommendation**

Databricks recommends file notification mode using file events instead of directory listing mode for most workloads, particularly continuous workloads. Directory listing is still useful for simple setups, smaller directories, one-time backfills, or environments where file events cannot be enabled.

Important nuance: even with managed file events, the first run performs a full directory listing to establish the stream position. Databricks also recommends running the stream at least once every seven days so the cached read position remains usable.

## 5. Schema Inference: How Auto Loader Learns the Columns

If you do not provide a schema, Auto Loader can infer one. On the initial read it samples up to 50 GB or 1,000 files, whichever limit is crossed first, and stores schema history under a _schemas directory inside the configured schemaLocation.

| Source format | Default inference behavior |
|---|---|
| JSON / CSV / XML | Columns are inferred as STRING by default to reduce type-conflict surprises. |
| Parquet / Avro | Uses types encoded in the source files and merges sampled schemas. |
| Text / binaryFile | Fixed schema; type inference is not applicable. |

If you want Spark-style type inference for JSON, CSV, or XML, you can enable cloudFiles.inferColumnTypes=true. For production pipelines, however, treating ingestion as a raw/bronze layer and parsing types downstream is often easier to govern.

**Schema inference enabled by schemaLocation**

```
df = (spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "csv")
  .option("cloudFiles.schemaLocation", schema_path)
  .option("header", "true")
  .load(source_path))
```

## 6. Schema Evolution: What Happens When a New Column Appears?

Schema inference answers “what is the schema when the stream starts?” Schema evolution answers “what should happen later when the incoming schema changes?” These are separate concepts.

| Evolution mode | Behavior |
|---|---|
| addNewColumns (default when schema is inferred) | Auto Loader records the new columns in schema metadata and the stream fails with UnknownFieldException. After restart, processing continues using the expanded schema. |
| addNewColumnsWithTypeWidening | Also supports selected type widening such as int to long. Public Preview on DBR 16.4+. |

| | |
|---|---|
| rescue | Does not evolve the schema; new columns are captured in the rescued-data column and the stream keeps running. |
| failOnNewColumns | Fails when new columns appear; schema is not updated automatically. |
| none | Does not evolve the schema. New columns are ignored unless rescuedDataColumn is configured. |

**Very common interview trap**

The default addNewColumns behavior is not "the stream never stops." Auto Loader updates the tracked schema and raises an error so the stream can be restarted with the new schema. Databricks recommends running such streams in Lakeflow Jobs so restart behavior can be automated.

# 7. _rescued_data vs _corrupt_record

These solve different problems and should not be treated as the same thing.

| Column | What it captures | Example |
|---|---|---|
| _rescued_data | Fields that do not match the expected schema: missing from schema, type mismatch, or case mismatch. | Schema expects amount BIGINT but a record contains amount="unknown". |
| _corrupt_record | An entire row that cannot be parsed according to the file format parser. | Malformed JSON or a structurally broken CSV record. |

When Auto Loader infers a schema, it automatically adds _rescued_data. The column contains a JSON blob with rescued fields plus the source file path. This prevents unexpected fields from being silently discarded.

**Production tip**

Track the count and percentage of rows with _rescued_data or _corrupt_record. These columns are data-quality signals, not just technical leftovers.

# 8. Checkpoints and Exactly-Once File Ingestion

Auto Loader stores discovered-file state and stream progress in the checkpoint. This state allows it to determine whether a file has already been processed. Databricks uses RocksDB-backed state for Auto Loader file tracking and provides exactly-once ingestion guarantees under the normal default behavior.

### Checkpoint is part of the pipeline state

```
checkpoint_path = "/Volumes/main/ops/checkpoints/orders"

query = (df.writeStream
    .option("checkpointLocation", checkpoint_path)
    .toTable("main.bronze.orders"))
```

- Use a separate checkpoint for each stream/source workload.
- Do not apply object lifecycle deletion policies to checkpoint directories. Deleting checkpoint state can corrupt the stream state and force a restart from scratch.
- Do not casually change checkpoint paths between runs; a new checkpoint means Auto Loader has no previous progress state.
- If cloudFiles.allowOverwrites=false (default), a discovered file is processed once. If overwrite processing is enabled, duplicate/ordering considerations become more complex.

# 9. What “Exactly Once” Does and Does Not Mean

Exactly-once here should be understood as file-ingestion state: Auto Loader remembers the files it has processed so that a normal stream restart does not re-ingest the same file. It does not magically make every downstream side effect exactly-once.

- Writing to Delta through Structured Streaming is designed to integrate with checkpoint-based progress.
- If your foreachBatch logic calls an external API, sends emails, or writes to a non-idempotent sink, you must design that side effect for retries yourself.
- If source files themselves are overwritten and allowOverwrites is enabled, version semantics need additional care.

# 10. Continuous vs Incremental-Batch Operation

Auto Loader uses Structured Streaming, but “streaming” does not necessarily mean a 24x7 cluster. You can run continuously or trigger a finite incremental batch.

| Pattern | When to use | Typical behavior |
|---|---|---|
| Continuous / ProcessingTime | Low-latency feeds where new files should be picked up repeatedly. | Cluster/query stays active and checks for work on the configured interval. |
| AvailableNow | Scheduled ingestion such as every 15 minutes or hourly. | Processes all files available at trigger time, then stops. |
| Once (legacy pattern) | Older finite-trigger pattern. | AvailableNow is generally the more capable modern choice. |

### Incremental batch with AvailableNow

```
(df.writeStream
   .trigger(availableNow=True)
   .option("checkpointLocation", checkpoint_path)
   .toTable("main.bronze.orders"))
```

This pattern is useful because you still get checkpointed incremental ingestion without paying for a continuously running stream when near-real-time latency is not required.

# 11. Production-Ready Example

### Example: bronze ingestion with managed file events

```
from pyspark.sql.functions import current_timestamp, input_file_name

source_path = "/Volumes/main/landing/orders"
schema_path = "/Volumes/main/meta/autoloader/orders_schema"
checkpoint_path = "/Volumes/main/meta/checkpoints/orders"

orders = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", schema_path)
    .option("cloudFiles.useManagedFileEvents", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load(source_path)
    .withColumn("_ingested_at", current_timestamp())
    .withColumn("_source_file", input_file_name()))

(orders.writeStream
    .trigger(availableNow=True)
    .option("checkpointLocation", checkpoint_path)
    .option("mergeSchema", "true")
    .toTable("main.bronze.orders"))
```

In a real production design, source, schema, and checkpoint paths should be environment-specific. Consider keeping schema and checkpoint metadata outside business-data landing folders so that operational metadata is not mixed with source files.

## 12. Auto Loader with Unity Catalog

Modern Databricks designs commonly use Unity Catalog external locations and volumes. Managed file events are configured at the external-location level, and Auto Loader streams reading from that location can use cloudFiles.useManagedFileEvents=true.

Databricks best-practice guidance recommends using volume paths for optimized file discovery with file events. A volume gives governed access to non-tabular files while Unity Catalog manages permissions on the storage location.

**Example governed source path**

```
/Volumes/<catalog>/<schema>/<volume>/incoming/orders
```

**Architecture connection**

Unity Catalog governs who can access the source files and target objects. Auto Loader solves the incremental-ingestion problem. Structured Streaming provides the execution model. Delta Lake provides the transactional table sink. These are different layers working together.

## 13. Performance and Cost Considerations

| Area | Why it matters | Practical guidance |
|---|---|---|
| File discovery | Repeated LIST operations grow with source-directory size. | Prefer managed file events for large/continuous production workloads. |
| Small files | Millions of tiny files add metadata and processing overhead. | Fix upstream file sizing where possible; avoid creating unnecessary tiny objects. |

| | | |
|---|---|---|
| Checkpoint state | Long-lived, high-volume streams accumulate file state. | Use supported newer runtimes and consider maxFileAge only with conservative settings. |
| Trigger frequency | More frequent runs can increase compute overhead. | Choose latency based on actual SLA, not "real-time by default." |
| Schema changes | Frequent uncontrolled changes cause restarts or rescued data. | Set ownership/contract expectations with producers and monitor schema drift. |

For long-lived high-volume streams, Databricks recommends Runtime 15.4 LTS or above for improvements in how Auto Loader starts with RocksDB state. If using cloudFiles.maxFileAge to cap state growth, Databricks recommends being conservative; overly aggressive expiry can allow old files to be reprocessed.

## 14. Common Mistakes

| Mistake | Why it is a problem |
|---|---|
| Reusing one checkpoint for multiple unrelated streams | Each stream/source workload should have its own checkpoint. Shared state can create incorrect progress tracking. |
| Deleting checkpoint files to "fix" a stream | You may destroy the file-history state and force reprocessing or a full restart. |
| Thinking _rescued_data means corrupt row | Rescued data is for schema mismatch; corrupt_record is for parser-level malformed records. |
| Assuming new columns are added with zero interruption | With default addNewColumns, Auto Loader updates the schema then fails so the stream can restart. |
| Using directory listing forever at very large scale | At large/continuous scale, listing the whole directory can drive latency and LIST API cost. |
| Assuming file arrival order is guaranteed | Neither file events nor listing guarantee business-event ordering. Handle late/out-of-order data downstream. |
| Turning on source cleanup with multiple readers | If multiple streams or tools need the same landing files, automated deletion can break other consumers. |

## 15. Decision Guide: Which Setup Should I Choose?

| Scenario | Suggested pattern |
|---|---|
| Small PoC or learning exercise | Directory listing + schemaLocation + checkpoint. |
| Hourly production ingestion | Auto Loader + AvailableNow + managed file events if available. |
| Near-real-time file arrival | Continuous trigger + managed file events. |
| Strict schema contract | Provide schema or hints; use failOnNewColumns or controlled evolution. |
| Messy upstream data | Keep _rescued_data; add _corrupt_record; quarantine/monitor bad rows. |
| Unity Catalog production architecture | External location/volume + governed checkpoint/schema metadata + managed file events. |

## 16. Auto Loader vs Normal Spark File Read

| Topic | spark.read | Auto Loader (spark.readStream + cloudFiles) |
|---|---|---|
| Intent | Read a set of files now. | Incrementally ingest files over time. |
| State | No persistent memory of previously read files by default. | Checkpoint tracks discovery/progress. |
| New-file discovery | Your scheduling/code decides what to read each run. | Built-in incremental discovery. |
| Schema history | No Auto Loader schema metadata. | schemaLocation can track inference/evolution. |

| | | |
|---|---|---|
| Streaming integration | Batch DataFrame. | Structured Streaming source. |
| Large-scale arrivals | You build more of the bookkeeping yourself. | Designed specifically for scalable file ingestion. |

# 17. Interview Questions & Answers

**1. What is Databricks Auto Loader?**

A Structured Streaming-based Databricks ingestion feature that incrementally discovers and processes new files from cloud object storage through the cloudFiles source.

**2. Is Auto Loader the same as Structured Streaming?**

No. Structured Streaming is the streaming execution framework. Auto Loader is a file-ingestion source built on top of it.

**3. What is the role of schemaLocation?**

It stores Auto Loader schema metadata and schema changes. It is different from checkpointLocation, which tracks streaming progress and file state.

**4. How much data does Auto Loader sample for schema inference?**

Up to the first 50 GB or 1,000 discovered files, whichever threshold is crossed first.

**5. What is the default schema evolution behavior when schema is inferred?**

addNewColumns. Auto Loader records the new columns, raises UnknownFieldException, and after restart the stream continues using the evolved schema.

**6. Difference between _rescued_data and _corrupt_record?**

_rescued_data captures fields that do not conform to the schema; _corrupt_record captures rows the parser cannot parse.

**7. Why are checkpoints critical?**

They store stream progress and file-tracking state that support recovery and exactly-once file ingestion.

**8. Directory listing vs file events?**

Directory listing scans the source directory for new files. File events use cloud notifications and cached metadata; Databricks recommends file events for most production workloads because they scale better.

**9. Can Auto Loader run as a scheduled batch?**

Yes. Use Structured Streaming with Trigger.AvailableNow to process currently available files and then stop.

**10. Does Auto Loader guarantee event ordering?**

No. File discovery/processing order is not guaranteed. Downstream logic should handle late or out-of-order data.

# 18. Practical Debugging Checklist

- Is the source path correct and readable by the compute identity?
- Does the stream have a unique, durable checkpointLocation?
- If schema inference/evolution is used, is schemaLocation durable and writable?
- If the stream failed after a new column, inspect the schema-evolution mode before treating it as a system failure.
- Inspect _rescued_data and _corrupt_record before discarding bad rows.
- If managed file events are enabled, verify the external location has working file-events permissions.
- If ingestion latency grows with directory size, check whether the workload is still using directory listing.

- Confirm the trigger type matches the SLA: continuous vs AvailableNow.
- Do not “reset” the pipeline by deleting checkpoints unless you intentionally accept reprocessing/rebuild consequences.

# 19. The Mental Model to Remember

### Auto Loader in one flow

```
NEW FILE ARRIVES
      |
      v
DISCOVER IT
(directory listing OR file events)
      |
      v
CHECK STATE
(checkpoint: have I processed this file?)
      |
      v
READ + PARSE
(cloudFiles.format + schema)
      |
      +--> mismatch? -> _rescued_data
      +--> malformed row? -> _corrupt_record
      |
      v
WRITE TO DELTA
      |
      v
SAVE PROGRESS IN CHECKPOINT
```

**One-line summary**

Auto Loader = scalable incremental file discovery + schema management + checkpointed ingestion, executed through Spark Structured Streaming.

# 20. Day 22 Quick Revision

| Remember this | Answer |
|---|---|
| API source | cloudFiles |
| Execution engine | Spark Structured Streaming |
| Primary job | Incrementally discover and ingest new cloud files |
| Initial schema sample | 50 GB or 1,000 files, whichever is crossed first |
| Schema metadata | cloudFiles.schemaLocation |
| Progress/file state | checkpointLocation |
| Unexpected schema fields | _rescued_data |
| Malformed parser rows | _corrupt_record |
| Recommended discovery for most production workloads | File notification mode using managed file events |
| Finite incremental run | Trigger.AvailableNow |

# 21. Sources & Verification Notes

Technical statements in this document were checked against current official Databricks documentation. Key references:

- Databricks Docs — Configure schema inference and evolution in Auto Loader (updated Sep 11, 2026).
- Databricks Docs — Compare Auto Loader file detection modes.
- Databricks Docs — Auto Loader with file events overview (updated Sep 11, 2026).
- Databricks Docs — Configure Auto Loader streams in directory listing mode (updated Sep 11, 2026).
- Databricks Docs — Configure Auto Loader for production workloads.
- Databricks Docs — Auto Loader best practices.
- Databricks Docs — Auto Loader FAQ.


Series context: this document expands the Day 22 reel topic “Autoloader — Naye Files Ko Khud-Ba-Khud Pakadna” into detailed engineering notes, including the reel’s focus on cloudFiles, schema inference, rescued data, and directory-listing versus file-notification discovery.